

TECHNICAL REVIEW

AsymptoticAnalysis

Incremental code audit, Wolfram Language comparison, and
Mathics compatibility

Prepared for Vladimir Reshetnikov

September 9, 2026, Pacific daylight time

Repository	VladimirReshetnikov/Asymptotic
Reviewed commit	7d1bc832895c
Mathics reference	Mathics3 10.0.1; SymPy 1.14.0
Review mode	Source analysis and independent executable oracles

Principal result. The most important new risks are in the semantic boundary between the package and Mathics, rather than in the already extensively reviewed power–log coefficient formulas. Two-argument Lambert functions can be misinterpreted during *exact symbolic conversion*; the empty-list workaround misses a public coefficient-query path; and first-match searches lose short-circuit behavior in the compatibility layer.

Execution boundary. Seven independent Python tests passed. No Wolfram or Mathics execution of the package, no full upstream suite, and no native performance benchmark were completed in this review. The accompanying kernel probes are explicitly unrun specifications, not test receipts.

Contents

1	Assessment and priorities	3
2	Snapshot, scope, and evidence discipline	4
2.1	What was actually inspected	4
2.2	Four different kinds of evidence	4
3	The mathematical and implementation model	5
3.1	Coordinates before coefficients	5
3.2	Why generalized inverse coefficients are useful	6
3.3	Specialized scales are different algebras	6
4	Comparison with built-in Wolfram Language	7
4.1	Compare contracts, not only function names	7
4.2	The order argument needs explicit alignment	8
4.3	Native output is not a failed analytic object	8
4.4	Discrete support needs a different approach set	9
5	Mathics implementation: substantial progress, bounded evidence	9
5.1	How the compatibility layer works	9
5.2	What the repository's validation does and does not establish	9
5.3	Compatibility should be reported in layers	10
6	F01: Lambert argument order is an exact symbolic defect	11
6.1	The precise bridge mismatch	11
6.2	Executed foreign-stage counterexamples	11
6.3	The numerical dispatcher has a separate limitation	12
6.4	A package-generated exposure, not only raw user syntax	12
6.5	Fix the representation and both conversion directions	13
7	F02: public empty multi-indices bypass the Map workaround	13
7.1	The workaround's actual coverage	13
7.2	A valid public argument reaches native Map	13
7.3	Narrow repair and wider inventory	14
8	F03: a portable case can succeed after a failed load	14

9 F04: default Limit direction changes the proposition being proved	15
10 F05: FirstPosition performs a complete Position search	16
10.1 Why the implementation is not operationally equivalent	16
10.2 A tempting repair is unavailable in the pinned interpreter	17
11 Two bounded advisories, not reproduced public failures	17
11.1 A01: dense polynomial conversion hidden inside a sparse adapter	17
11.2 A02: interrupted loading can bypass context cleanup	17
12 An executable, exact-rational oracle for the lower Lambert branch	18
12.1 Replace the ill-behaved conversion boundary with a monotone equation	18
12.2 A rational logarithm enclosure	18
12.3 Executed checks and limitations	19
13 Development recommendations that follow from the new findings	20
13.1 Treat compatibility adapters as a small semantic runtime	20
13.2 Add mutation tests at the semantic boundary	20
13.3 Record semantic state at value boundaries	20
13.4 Budget observable work, not only final support	21
13.5 Sequence the work with explicit acceptance gates	21
14 Novelty crosswalk and exclusions	21
15 Reproduction guide and archive contents	22
16 Conclusion	23

1 Assessment and priorities

The repository has developed into a substantial asymptotic-calculus package, not merely an inverse-series formula generator. Its central abstraction is a finite generalized expansion accompanied by a remainder description, a source/target coordinate convention, retained assumptions, and enough provenance to support selected refinement operations. It combines exact power-log jets with specialized logarithmic, Lambert, Gamma, Barnes, Fourier, exponential, and flat-sector engines. Ordinary arithmetic can preserve the expansion object rather than immediately discarding its error information. Explicit native backends occupy a separate result contract. These are meaningful architectural distinctions, especially for real irrational exponents and inverse problems with logarithmic coefficients. [R1, R7, R8, R9]

The current snapshot must not be judged as though the first review’s bugs were still present. The repository indexes 27 prior review packages across three waves and 167 attributed entries *before deduplication*; these numbers are not counts of current defects. Its implementation register and wave-3 intake already record numerous correctness repairs and unresolved design issues. This article uses those registers as the exclusion list, and concentrates new findings on the recently added Mathics adaptation layer and the precise runtime boundaries that it creates. [R3, R4, R5]

My overall assessment is therefore two-part. On the Wolfram kernel, the package is a technically interesting extension with a deliberately narrower real-analytic contract than the complete built-in asymptotics system. Existing unresolved findings should remain tracked under their existing identifiers. On Mathics, the implementation is much more developed than a superficial compatibility shim, but the present evidence does not establish complete compatibility. More importantly, a conservative symbolic admission policy does not by itself protect values that later cross an incompatible special-function bridge.

The immediate order of work should be: isolate nonprincipal Lambert expressions from the defective bridge; cover caller-owned empty lists in public APIs; make load success an invariant of the portable test runner; restore first-match behavior on the actual hot search shape; and make implicit limit direction impossible in proof-critical adapter calls. The sparse-conversion and interrupted-loading issues discussed later are bounded advisories rather than newly reproduced public failures.

ID	Finding	Priority	Evidence boundary
F01	Nonprincipal ProductLog: exact symbolic bridge reverses argument roles	High	Bridge source + executed SymPy stage; package call unrun
F02	Empty-list workaround excludes a public multi-index validation path	High	Missed call site proved from source; crash probe unrun
F03	Portable case selection can report success after Get returned \$Failed	Medium	Runner/suite control-flow proof; fault-injection probe unrun
F04	Adapter default Limit silently means from below	Medium, bounded	Primitive contract mismatch; no public wrong result established
F05	FirstPosition collects every match and runs all predicates	Medium	Algorithm and consumer identified; no kernel timing claim

ID	Finding	Priority	Evidence boundary
A01	Sparse coefficient extraction becomes dense in polynomial degree	Advisory	Adapter allocation structure; public stress route not established
A02	Compatibility-context cleanup occurs only on successful loading	Advisory	Missing exceptional cleanup; interruption tests unrun

2 Snapshot, scope, and evidence discipline

2.1 What was actually inspected

All repository claims in this article refer to 7d1bc832895cc90a9b2a978b7b7684acab908bd2. The commit timestamp is September 10, 2026 at 02:10:22 UTC, or September 9 at 19:10:22 Pacific daylight time. The date difference is a timezone conversion, not a second source snapshot. Mathics implementation claims are pinned separately to version 10.0.1 at f6a987e704313e37b2b46ed4fe0b61aab108b58a. [R2, M1]

Inspection covered the repository and module indexes, the consolidated implementation register, the complete wave-3 intake, an original wave-3 review’s own evidence description, the core jet and object implementation, native delegation, representative inverse and Fourier engines, all fifteen named Mathics compatibility modules encountered in the loader, compatibility documentation, the portable suite’s entry and foundational cases, its Python runner, and its CI workflow. Large sources were read in bounded ranges. This is a targeted incremental audit across the architecture; it is not a claim that every line of every mathematical engine or all 27 historical articles was independently reread. [R8, R4, R5, R6, R13, R21, R22, R23]

The exclusion process was deliberately wider than the new code inspection: the maintained register and intake consolidate the historical finding inventory, so an old issue does not become “new” merely because it was rediscovered through another call path. The novelty crosswalk in Section 14 makes the closest overlaps explicit. Historical reports are used as records of what has already been said, not as evidence that an old defect persists in the present commit.

2.2 Four different kinds of evidence

Source-confirmed means that the relevant definition, dispatch choice, or allocation strategy is visible in the pinned code. For example, an installer that enumerates only private symbols demonstrably does not rewrite a public definition. It does not follow that every execution of that public definition crashes.

Independent execution means that code in this archive ran outside the Wolfram/Mathics package. In particular, SymPy 1.14.0 was exercised directly with the exact argument tuples that the Mathics bridge supplies, and an independent rational Lambert enclosure algorithm was implemented and tested. These results establish the foreign-library stage and the mathematical reference values, not an end-to-end interpreter trace.

Predicted public manifestation means a concrete package call is derived from source and supplied

as a regression specification, but was not run. Such a call is useful for fixing and testing the issue, but its predicted output is not mislabeled an observed package result.

Repository-reported validation means a receipt or status document maintained by the repository reports a result. Its commit, scope, and kernel remain attached to the claim. It is never merged into this review's independent test count.

A native Wolfram evaluator request failed at the service layer, and the working environment did not have a usable Mathics installation. Consequently, no complete native comparison, package benchmark, or Mathics acceptance run was obtained. This limits coverage, but does not erase a source-level argument-order defect or an independently checkable mathematical counterexample.

3 The mathematical and implementation model

3.1 Coordinates before coefficients

The basic real approach is represented through a positive local variable $u \rightarrow 0^+$. At a finite endpoint it is a signed displacement; at an infinite endpoint it is a reciprocal magnitude. This removes ambiguity from powers u^α for real α and allows logarithms to be handled as real quantities. The original source endpoint, selected side, inverse target endpoint, and observable power still matter: a positive uniformizer is not permission to erase the original branch. [R7, R10, R11]

An ordinary jet is a sparse list

$$T = \{(\alpha_1, P_1(L)), \dots, (\alpha_m, P_m(L))\}, \quad L = \log u,$$

representing $\sum_j u^{\alpha_j} P_j(\log u)$. The exponents are exact real numbers; the coefficients in the ordinary layer are polynomials in L . Internal sorting and merging must distinguish mathematical equality of exponents from merely identical syntax. The current core includes grouped exact-weight normalization and cutoff-aware sparse multiplication, so those earlier issues are not new findings here. [R7, R4]

The precision-tracked form is a triple

$$(T, P, D) \rightsquigarrow \sum_{(\alpha, Q) \in T} u^\alpha Q(\log u) + \mathcal{O}(u^P(1 + |\log u|)^D).$$

A finite P is a retained error frontier, not an extra known coefficient. The first ordering is by power: for fixed $a > 0$ and finite d , $u^a |\log u|^d \rightarrow 0$. Logarithmic degree matters when two errors share a power. This explains why complete blocks, rather than individual scalar monomials in the displayed answer, are the useful unit of many package operations.

For example,

$$A = u^a A_0(\log u) + \mathcal{O}(u^P |\log u|^D), \quad B = u^b B_0(\log u) + \mathcal{O}(u^Q |\log u|^E)$$

produces cross-errors with powers $P + b$ and $Q + a$, with logarithmic degrees augmented by the opposite leading polynomial. Cancellation of the displayed leading terms must not cancel an unknown remainder. This is background for interpreting the representation, not a reissued finding about the already reviewed precision algebra.

3.2 Why generalized inverse coefficients are useful

For the elementary test family

$$f(x) = x + x^{1+\alpha}, \quad \alpha > 0,$$

write $g(y) = y v(y^\alpha)$ near $y = 0^+$. Direct generalized reversion gives

$$\begin{aligned} g(y) = & y - y^{1+\alpha} + (1 + \alpha)y^{1+2\alpha} \\ & - \frac{(1 + \alpha)(2 + 3\alpha)}{2}y^{1+3\alpha} + \dots \end{aligned}$$

More generally, the coefficient of $y^{1+n\alpha}$ is

$$c_n = \frac{(-1)^n}{n!} \prod_{j=1}^{n-1} (1 + n\alpha + j), \quad n \geq 1.$$

The empty product at $n = 1$ is one. These formulas make sense without forcing α onto a rational lattice. When several correction gaps are present, different multi-indices can yield the same exponent; complete-block collection is then mathematically necessary, not only a display preference.

Logarithmic coefficients add a different requirement. For

$$f(x) = x + x^2 \log x,$$

letting $L = \log y$, substitution yields

$$\begin{aligned} g(y) = & y - y^2 L + y^3 (2L^2 + L) \\ & - y^4 \left(5L^3 + \frac{11}{2}L^2 + L \right) + \dots \end{aligned}$$

The accompanying Python check verifies that $g + g^2 \log g - y$ has no terms of degree below five in the formal ring, using $\log g = L + \log(g/y)$. It is a coefficient oracle, not a proof that an arbitrary imported formal series has an analytic remainder. The package's coefficient-polynomial machinery is a natural way to express this calculation.

3.3 Specialized scales are different algebras

The code does not force every problem into the same ordinary power-log representation. Lambert inverses extract an exact leading expression and use a small reciprocal-logarithmic variable for a correction bracket. Exact-core perturbation retains an inverse core while expanding by marker degree. Gamma and Barnes inverses use specialized core and logarithmic coordinates. Fourier coefficients are finite sums of modes in $\log u$, with a source-power grading separate from frequency. Flat sectors carry an exponential scale in addition to an inner algebraic jet. [R10, R11, R12, R8]

These distinctions should remain visible in a comparison. A marker-degree truncation is not automatically sorted by target exponent. A frequency budget is not a license to truncate frequency support silently. A relative error on a bracket multiplied by a large exact prefactor is not the same as an absolute error on the complete expression. The current usage strings and object metadata already expose much of this structure. The outstanding issue in this review is not that all of these contracts should be redesigned, but that the Mathics boundary must preserve them after construction as well as during construction.

4 Comparison with built-in Wolfram Language

4.1 Compare contracts, not only function names

The relevant built-in baseline is broader than `Series`. Wolfram documents Taylor, Laurent, selected fractional-power and logarithmic expansions, expansion at infinity, successive multivariate expansions, coefficient extraction, composition, and series reversion. `Asymptotic` supplies a broader asymptotic interface, while separate functions address equations, integrals, sums, products, differential equations, recurrences, and discrete limits. [W1, W3, W4, W6, W5]

The package is strongest where a real generalized inverse is represented as an inspectable object with its own block ordering, retained assumptions, remainder transport, and refinement recipe. It should not be described as the first system able to handle logarithms or non-Taylor asymptotics. Conversely, returning an ordinary native expression is not the same interface as maintaining a package-specific analytic error contract through subsequent operations.

Area	Built-in baseline	Package role and boundary
Ordinary local series	<code>Series</code> , <code>SeriesData</code> ; ordinary arithmetic, coefficients and composition	Considerable overlap. Package adds its own real-domain and error metadata. [W1, W2, R7]
Rational fractional powers	<code>SeriesData</code> explicitly represents a rational exponent lattice	Package supports exact real weights without requiring one rational denominator. This is a representation distinction, not a claim that every native irrational-power expression fails. [W2, R7]
Logarithmic coefficients	Native series can contain logarithmic or slower-growing coefficient expressions	Package treats complete polynomial-logarithmic blocks and an explicit logarithmic remainder degree. [W2, R7]
Inverse series	<code>InverseSeries</code> operates on native series; other solvers cover broader equations	Package selects real endpoint branches and supports specialized generalized inverse scales. [W3, R7, R10]
General asymptotics	<code>Asymptotic</code> and the related built-in family	Package can delegate selected requests, but native output remains a distinct contract. [W4, R9]
Exponential and special-function structure	Native functions may retain rapidly varying factors outside a <code>SeriesData</code> object	Package provides explicit prefactor/correction and inverse-core structures for selected families. [W2, R8]
Algebra on results	Native series support algebra and composition within their representation	Package transports its remainder descriptors through selected operations and keeps source/provenance information. [W2, R7]

Area	Built-in baseline	Package role and boundary
Numerical inverse assessment	Native numerical solvers supply roots and numerical diagnostics	Package distinguishes numerical comparisons from its restricted exact-rational interval certificates. Neither should be confused with the other. [R7, R13]
Discrete asymptotics	DiscreteAsymptotic and discrete limit functions	Not interchangeable with a continuous real germ. Native delegation currently centers on Series and Asymptotic. [W5, R9]
Breadth of mathematical problems	Built-in asymptotics include equations, integrals, products, sums, ODEs and recurrences	These are not all implemented by the package's own analytic engines. This is a scope comparison, not a newly discovered missing-feature list. [W6, R8, R5]

4.2 The order argument needs explicit alignment

For an ordinary Taylor example, the following two specifications are intended to retain the same powers:

```
AsymptoticExpansion[Exp[x], {x, 0, 4}, "Backend" -> "Package"]
Series[Exp[x], {x, 0, 3}]
```

The package cutoff is exclusive: retain powers less than four. The native order in the second call requests terms through degree three. Native SeriesData itself has its own exclusive exponent boundary internally; that does not make the public Series order argument identical to the package cutoff. For irrational weights or a prefactor/bracket representation, comparing only the integer arguments can be still more misleading. [R7, W1, W2]

Similarly, a requested number of package blocks is not necessarily the number of visible summands after expanding every logarithmic polynomial. A fair differential test first normalizes coordinate, branch, retained scale, and order convention, then compares coefficients and any compatible remainder statements. The earlier reviews already requested native differential testing. The additional point here is that an executable comparison must encode this alignment, rather than infer success from syntactically similar answers.

4.3 Native output is not a failed analytic object

The native path stores the native result and the held request rather than pretending that every returned expression carries the package's analytic remainder semantics. Several operations explicitly reject native objects when they require the package's analytic contract. That separation is correct in principle. It also means that an inert Mathics System`Asymptotic symbol is not an implemented native backend just because the name exists. [R9, R13]

One must distinguish three outcomes: a computed native expression, an unresolved native call, and a package failure with a mathematical or representational reason. A result object containing an un-

resolved call is not evidence that the requested asymptotic expansion was obtained. This audit does not repeat the existing native-status and option-dispatch findings; their historical identifiers should remain authoritative. [R4, R5]

4.4 Discrete support needs a different approach set

A useful concrete design consequence of the built-in comparison is the approach set. The expressions $\sin(\pi n)$ and 0 agree for integer n , but $\sin(\pi x)$ is not asymptotically zero on the continuous real ray. Thus a future discrete adapter cannot safely be represented by merely changing the backend label on a continuous expansion object. Its contract needs the approach set or filter, for example positive integers, possibly with residue-class data.

This also prevents a tempting confusion with the existing Fourier engine. Frequencies in $\log u$ are not ordinary integer-sequence frequencies modulo 2π . The new design requirement is to make the domain of approach an explicit conversion precondition. It is a sharper elaboration of the already recorded discrete/native extension opportunity, not a new claim that discrete asymptotics had been overlooked. [W5, R4, R12]

5 Mathics implementation: substantial progress, bounded evidence

5.1 How the compatibility layer works

The main loader conditionally loads Mathics-only support. An adapter context is inserted while package definitions are parsed, so selected names resolve to package-owned alternatives. The adapter context is removed from the public context path after normal loading. Other late adapters rewrite particular package definitions or install Mathics-compatible arithmetic rules. The official Wolfram path does not intentionally replace its built-ins with these alternatives. [R7, R14, R18, R17]

The implementation addresses genuine evaluator differences: scoped returns, association operations, held extraction, exact assumptions, simplification, finite derivative sums, negative-index assignment, empty-list mapping, and evaluation order in association construction. It also supplies selected defining-series expansions and a finite Stirling model where native support is missing. Some private proof attempts receive larger time budgets on Mathics, while explicit user deadlines are meant to retain their documented meaning. [R13, R14, R15, R20]

The good design decision is locality: the workaround belongs to the package, not to global mutation of System definitions. The less obvious cost is that locality creates a coverage problem. A rewrite restricted to private DownValues does not reach public DownValues, compatibility-context functions, SubValues, or user-facing expressions that are evaluated later. F01 and F02 are concrete consequences of that boundary, not arguments for globally patching the interpreter.

5.2 What the repository's validation does and does not establish

The compatibility document explicitly states that complete compatibility is a goal and is not yet achieved. It lists focused examples in loading, ordinary inverse/forward calculations, arithmetic, retained refinement, held callables, selected special functions and inverse scales, exact rational certificates, and numerical smoke tests. It also says that a consolidated run of the final source snapshot

is pending. [R13]

The same document reports a Wolfram preservation run with 1,452 passes and the same 12 existing failures across 79 suites, matching 1,464 records against the untouched 6687962 baseline on Wolfram 15.0.1 for Windows. Later definition comparisons use later controls and compare 2,051 modular and 2,050 standalone package symbols, together with selected built-in state and behavior probes. The document carefully avoids relabeling those later definition comparisons as a complete rerun of the earlier test suite. This review preserves that distinction. [R13]

The portable CI workflow is materially stronger than a single smoke script: it runs both modular and standalone entry points, divides cases into five groups of suites, starts fresh kernels for individual cases, and uses operating-system timeouts. The Python runner checks result markers, case identity, return codes, source stability, and diagnostics. These protections deserve credit. F03 is a narrow missing load invariant in this otherwise purposeful runner, not a claim that it has no validation discipline. [R23, R22, R21]

5.3 Compatibility should be reported in layers

A useful acceptance matrix separates the following layers. This is an interpretation of the current implementation and a proposed reporting structure, not a claim that all cells have passed.

Layer	Current evidence or mechanism	Remaining acceptance question
Load/evaluator	Scoped adapters; load/reload examples; modular/standalone paths	Is cleanup safe under interruption and partial failure?
Exact coefficient algebra	Selected polynomial, irrational-power and logarithmic examples	Do empty supports, aliased lists and symbolic-equality searches preserve semantics and budgets?
Proof/admission	Conservative realness and sign rules; selected affine and polynomial branch proofs	Are unresolved predicates distinguished from disproved ones, with explicit approach directions?
Special functions	Selected defining-series and Stirling adapters; principal Lambert normalization	Are all symbolic and numeric conversion directions branch-correct?
Operations and refinement	Focused arithmetic, flat, Fourier, and refinement examples	Does every supported composition of operations remain inside the admitted contract?
Numerical specialization	Limited principal Lambert and rational-root smoke evidence	Does evaluating a previously valid object cross an unsafe foreign-function bridge?
Certificates	Restricted exact rational examples and history updates	Do final numerical certificates use independent exact bounds, not a misleading backend value?
Presentation	Output-oriented workarounds and structured inspection advice	Does formatting preserve the object without executing or changing its meaning?

The proper present description is *partial, actively engineered compatibility with identified supported examples*, not “loads successfully, therefore compatible” and not “Mathics cannot be used at all.” Unsupported proofs should fail conservatively; supported numerical specializations must also avoid silently returning a value for a different mathematical function.

6 F01: Lambert argument order is an exact symbolic defect

6.1 The precise bridge mismatch

Wolfram uses `ProductLog[k, z]` for branch k at argument z . SymPy uses `LambertW(z, k)`. The retrieved Mathics 10.0.1 `ProductLog` class inherits the generic SymPy conversion behavior, sets its SymPy name to `LambertW`, and does not override argument preparation or reverse conversion. The base `prepare_sympy` returns the elements unchanged, and the generic conversion calls the target function with that unchanged tuple. The generic reverse conversion likewise returns the elements without restoring Wolfram order. [M2, M3, S1]

Thus the conversion maps

$$\underbrace{\text{ProductLog}(k, z)}_{\text{Wolfram: branch, argument}} \mapsto \underbrace{\text{LambertW}(k, z)}_{\text{SymPy: argument, branch}},$$

when it should map to `LambertW(z, k)`. A displayed round trip can conceal the problem: preserving the same printed tuple is not preserving the meaning of the tuple while it is inside the foreign system.

The repository’s compatibility documentation warns about incorrect argument order in the two-argument evaluator and says nonprincipal numerical evaluation remains unsupported. That warning is useful but too narrow. The defect exists before numerical evaluation: exact symbolic conversion already changes which argument is the branch selector. [R13, R17]

6.2 Executed foreign-stage counterexamples

The independent checks ran the following exact calls in SymPy 1.14.0:

```
LambertW(0, E)      # unchanged tuple for ProductLog[0,E]
LambertW(E, 0)      # correctly reordered tuple
LambertW(-1, -1/10) # unchanged tuple, using exact Rational(1,10)
LambertW(-1/10, -1) # correct branch -1 at argument -1/10
```

The first pair gives $-\infty$ versus 1. For the second pair, numerical evaluation of the exact SymPy expressions gives approximately

$$\begin{aligned} \text{LambertW}(-1, -1/10) &= -0.3181315052047641 + 1.3372357014306894i, \\ W_{-1}(-1/10) &= -3.5771520639572972. \end{aligned}$$

These numbers are recorded in `evidence/independent-checks.json`. They are results of the foreign-library stage, not captured Mathics outputs.

The distinction extends to differentiation. Differentiating `LambertW(-1, y)` with respect to y asks for a derivative in the branch slot and remains a derivative object in the independent check. Differentiating

the correct $\text{LambertW}(y, -1)$ gives the ordinary analytic derivative

$$\frac{d}{dy} W_{-1}(y) = \frac{W_{-1}(y)}{y(1 + W_{-1}(y))}.$$

This demonstrates why checking one floating-point value does not exhaust the affected contract. Simplification, differentiation, and other symbolic consumers must see the same mathematical argument convention.

6.3 The numerical dispatcher has a separate limitation

The base Mathics `MPMathFunction` has `nargs = {1}`. The retrieved `ProductLog` class does not override this. In the inexact path, the base numeric function lookup rejects unsupported arities; in the exact path it first tries SymPy conversion. Consequently, the description “the two-argument numeric evaluator always computes the wrong number” is also too crude. A two-argument inexact request may remain unresolved, while an exact request can already have been misinterpreted by SymPy. A repair must address both paths. [M2, M3]

6.4 A package-generated exposure, not only raw user syntax

The package’s late core adapter normalizes the principal branch to the safe one-argument `ProductLog[z]` form. For nonzero branches it passes through `System`ProductLog[branch,z]`. It rewrites selected inverse-core consumers, but it does not install a general branch-safe value representation. [R17]

Consider the real function

$$f(x) = -x \log x, \quad 0 < x < e^{-1}.$$

It is strictly increasing there because $f'(x) = -\log x - 1 > 0$, and its small- x inverse is

$$x(y) = \exp(W_{-1}(-y)) = -\frac{y}{W_{-1}(-y)}, \quad 0 < y < e^{-1}.$$

The exact-core constructor recognizes this affine-logarithmic form. With no perturbation, it should return the exact selected core rather than an approximate correction. A focused public regression is:

```
s = AsymptoticCoreInverse[-x Log[x], 0, {x, 0}, {y, 0}];
v = N[s[1/10], 40];
(* Require a real v, 0 < v < 1/E, and -v Log[v] == 1/10
   to the requested numerical tolerance. *)
```

The source path is `corePerturbationAutomaticInverse`, through the Mathics core adapter, into the expression stored in a `GeneralizedSeries` object. Numeric application of an ordinary object substitutes into its expression directly. There is no branch-safe specialization gate in that object application definition. [R11, R17, R7]

This end-to-end call was not run here. The source nevertheless identifies a concrete crossing: a package-generated nonprincipal expression can leave the constructor through a generic substitution/evaluation boundary. Calling nonprincipal evaluation “unsupported” does not make this crossing fail closed.

6.5 Fix the representation and both conversion directions

An upstream Mathics repair should reorder two-argument input before SymPy conversion, reverse that reordering on SymPy output, admit the correct two-argument numerical arity, and call mpmath with its own (z, k) ordering. It must preserve one-argument principal expressions and validate the branch convention. A patch that changes only `prepare_sympy` is incomplete: the reverse bridge would still print or reconstruct the tuple in the wrong Wolfram order.

For the package, the immediate containment strategy is a package-owned opaque representation for unsupported nonprincipal branches, or an explicit failure at the supported specialization boundary. The opaque head must have deliberate derivative and evaluation rules; it must not be translated back to raw System`ProductLog before a known-unsafe simplifier sees it. Globally changing Mathics's System definitions is not necessary and would undermine the locality of the existing design.

The regression matrix should include both real branches, exact and inexact arguments, symbolic differentiation, forward/reverse conversion, special values, behavior near $-1/e$, and evaluation after storage in a result object. Include an independent branch oracle; a round-trip test that compares only syntax can pass while both conversion directions preserve the same wrong interpretation.

Section 12 supplies a concrete independent W_{-1} oracle based only on exact rational logarithm bounds. It is usable immediately for testing and as a reference for a future package-owned evaluator. It is not claimed to be an already integrated Mathics repair.

7 F02: public empty multi-indices bypass the Map workaround

7.1 The workaround's actual coverage

The repository documents a Mathics 10.0.1 defect in which mapping over a reused empty list can invalidate internal cached list properties, with a later nested numeric list raising a Python assertion. Its minimal example is an empty list, a map over that list, then reuse of that same list. The package installs `mathicsMap`, which returns an empty list directly in the exact two-argument empty-list case and delegates other cases to the native map. [R18, R19]

The installation pass enumerates only names in `AsymptoticAnalysis`Private`*`` and rewrites their `DownValues`. This is a precise, source-confirmed limitation. It does not rewrite the public context or the separate `AsymptoticAnalysis`Mathics`` context. Its documentation even describes the public-definition exclusion explicitly. That locality is not inherently wrong, but it is insufficient when a public definition itself contains the dangerous map.

7.2 A valid public argument reaches native Map

The public definition of `InverseExpansionCoefficient[model,k]` validates the multi-index using a mapped integer predicate:

```
If[Length[k] != Length[model["Gaps"]] ||
  !(And @@ (IntegerQ[#] && # >= 0 & /@ k)), ...]
```

This `DownValue` belongs to the public `InverseExpansionCoefficient` symbol, not to a private helper.

Therefore the installer leaves its Map untouched. [R7, R18]

The empty multi-index is not malformed input. A monomial model has no correction gaps, so its zero-dimensional coefficient query legitimately uses {}. The focused regression is:

```
model = PowerLogModel[x, {x, 0}];
k = {};
c = InverseExpansionCoefficient[model, k];
{k, 2, 0}
```

The coefficient should be one and the final nested list should remain safe. The current validation evaluates the native map on the caller's empty list before entering the private coefficient machinery. The exact later crash location depends on evaluator aliasing and reuse, and was not observed in a kernel here. The missed dangerous operation is established; the supplied test determines its public manifestation.

7.3 Narrow repair and wider inventory

For this call site, the least intrusive change is an empty fast path:

```
If[Length[k] != Length[model["Gaps"]] ||
  !(k == {} || And @@ (IntegerQ[#] && # >= 0 & /@ k)), ...]
```

The length condition still rejects an empty index for a model with nonempty gaps. The short-circuit disjunction avoids mapping only where mapping has no work to do. It preserves the ordinary nonempty validation.

The compatibility context has its own mapped lookup and key-selection helpers, also outside the private-only installer. They warrant the same call-site inventory. This does not justify a blind rewrite over arbitrary held payloads, object OwnValues, or all System definitions. Instead, maintain an explicit inventory of executable package-owned definitions and the value tables in which they live, and choose either local empty fast paths or an explicitly qualified safe mapper.

The missing test dimension is *post-call reuse*. A test that checks only c["Coefficient"] may overlook damage to an argument that is reused afterward. A stronger contract probe checks both the returned value and the usability of caller-owned inputs. This is especially relevant to a symbolic system whose expression implementation uses shared objects internally.

8 F03: a portable case can succeed after a failed load

The portable suite records

```
portableLoadResult = Check[Get[portableSource], $Failed];
```

but does not immediately terminate when this value is \$Failed. One selected case, loading-no-messages, checks it. Other selected cases can execute and emit their own success records regardless of the load result. The Python runner correctly checks the selected result record, but it cannot infer a load invariant that the suite never emits or enforces. [R21, R22]

A minimal fault-injection fixture is a valid text file whose content is simply `$Failed`. The runner accepts it as an existing source file. Select the case `primitive-check-is-unpolluted`, whose tested expression is `Check[1+1,$Failed]` and whose expected value is two. That primitive can succeed even though `Get` explicitly returned failure. The bundle includes this fixture and the exact invocation recipe. The injection was not executed in Mathics here; the conclusion follows from the suite's independent top-level expressions.

This is not a claim that the entire suite would pass after a failed load. The dedicated loading case should fail, and many package-dependent cases would fail or remain unresolved. The issue is that a focused case receipt can be labeled successful without its declared package setup succeeding. For debugging and incremental acceptance, a “pass” should imply that all required setup phases passed.

The narrow fix is an unconditional load gate, placed as a separate top-level expression immediately after the existing load:

```
If[portableLoadResult === $Failed,
  Print["ASYMPTOTIC_PORTABLE_LOAD_ERROR"];
  Exit[2]];
```

The existing runner treats exit code two as an error, so this already prevents success. A dedicated `LoadError` outcome would make diagnostics clearer. A small required-symbol check can also catch a nominally successful source that did not install the package, but the gate should not assume every legitimate `Get` returns `Null`.

The separate-expression placement matters. The repository documents a Mathics interaction in which a previous `Print` in the same input evaluation can pollute `Check`. Diagnostics must follow the checked setup rather than be inserted into the same evaluation before it. [[R13](#), [R21](#)]

Negative-result tests need a complementary discipline. `TrueQ[unresolved]` is false; therefore “the prover did not establish a forbidden statement” can be true even when a helper is missing. Pair negative examples with positive controls and, where the contract requires a decision, record whether the underlying result was `True`, `False`, `unresolved`, or an explicit failure. This is an additional test-design consequence, not a claim that the chosen minimal failed-load fixture makes every negative case pass.

9 F04: default Limit direction changes the proposition being proved

The Mathics simplification adapter declares its own `Limit` options with `Direction -> Automatic`, then translates `Automatic` to the numeric direction one, alongside the intended string translations from above and below. The result is a from-below limit whenever this adapter is called without an explicit direction. [[R15](#)]

Current Wolfram documentation gives `Reals` as the default direction for ordinary real two-sided limits. This distinction is mathematically observable on

$$h(t) = \frac{|t|}{t}, \quad t \rightarrow 0.$$

The limit from below is -1 , the limit from above is 1 , and the two-sided limit does not exist. A local adapter replacing a two-sided request by a one-sided value has changed the proposition, not merely the syntax of its options. [[W7](#)]

The proposed primitive regression compares the adapter’s default request with the two-sided contract. It is explicitly a *primitive* compatibility issue: this review did not establish a public constructor that reaches this adapter with an omitted direction and then returns an incorrect expansion. Many proof-critical package calls may already specify a positive local approach. No blanket claim that source directions are reversed is warranted.

There are two defensible fixes. One is to require every internal proof limit to carry an explicit approach direction, enforced by a source audit and regression inventory. The other is to implement the two-sided adapter contract: compute both sides independently and return a common result only after the required equality is established; differing sides imply no two-sided limit, while unresolved sides must remain unresolved. Infinite endpoints and parameter assumptions require their own admissible-direction checks rather than a universal numeric replacement.

This finding is distinct from earlier source-versus-target direction metadata issues. It concerns a compatibility primitive’s default semantics. The strongest practical recommendation is to make omission of direction impossible where a limit is being used as proof evidence, even when the present call sites happen to be safe.

10 F05: FirstPosition performs a complete Position search

10.1 Why the implementation is not operationally equivalent

The adapter implements FirstPosition by calculating

```
positions = Position[expr, pattern, levels, Heads -> heads];
If[positions === {}, ReleaseHold[default], First[positions]]
```

This yields the first member of the correct traversal’s result list, but it searches the entire expression and materializes every matching position before returning. The issue is not traversal order. It is loss of short-circuiting and the execution of predicates that a first-match search need not reach. [R14, M4]

The package has a direct consumer: symbolic-mode jetMerge searches existing output buckets using a pattern condition involving symbolicEqualQ. That predicate may call symbolic simplification. Repeated terms that belong to the first bucket should be cheap to locate; the adapter instead tests the remaining buckets as well. [R7]

Suppose m distinct buckets have already been built and r subsequent rows all match the first bucket. On this workload, the first-match part of a short-circuit search uses approximately r candidate predicates, while the complete-position adapter uses rm . This is not a claim that every input changes asymptotic complexity or that all symbolic merges are linear. It isolates the adapter-induced factor on a concrete workload with an early match. The archive records an operation-count model, not measured Mathics wall-clock timings.

Predicate evaluation count can also matter semantically for effectful pattern tests, although the package’s equality predicates should ordinarily be treated as mathematical computations. Expensive but pure predicates alone are sufficient to make this a performance defect worth fixing.

10.2 A tempting repair is unavailable in the pinned interpreter

In a system with the corresponding native form, asking `Position` for at most one match could preserve the desired behavior. However, the retrieved Mathics 10.0.1 `Position` implementation exposes an optional level specification and options, walks all selected levels, and appends every match; it does not implement the maximum-count form as a separate argument. Therefore simply changing the call to `Position[... , 1, ...]` is not a verified repair for this target. [M4]

A bounded specialization is more appropriate. The observed hot consumer uses a list, level specification `{1}`, and `Heads -> False`. A scan with a private unique catch tag can stop on the first match, then evaluate the held default only when no match exists. The archive contains a candidate definition and a predicate-count regression. Other level and head forms can retain the existing generic traversal until a fuller short-circuit walker is implemented and tested.

Installation itself must respect Mathics's documented behavior when defining a replacement left-hand side while an old definition exists. Install the specialization and generic fallback from clean definitions during loading; do not assume an interactive late overwrite is automatically harmless. The code proposal is unrun in a kernel and is labeled accordingly.

11 Two bounded advisories, not reproduced public failures

11.1 A01: dense polynomial conversion hidden inside a sparse adapter

The Mathics algebra adapter's univariate coefficient extraction uses `CoefficientList` and then selects nonzero entries. That can turn a sparse polynomial into a degree-sized intermediate. For $1 + x^D$, the useful support has size two, but the intermediate coefficient list has $D + 1$ slots. This follows from the representation choice, independently of benchmark hardware. [R16]

This article does not establish a public package request with arbitrarily large D that reaches this exact adapter before a different budget or simplification path stops it. It therefore does not claim a reproduced denial of service or attach a speculative byte count. The advisory is a boundary audit: identify callers of the compatibility coefficient extractor, record their degree and support bounds, and avoid forcing sparse expressions through a dense representation when the output contract is sparse.

A safe sparse converter would distinguish a compact monomial x^D from a genuinely dense expansion such as $(1 + x)^D$. Multiplication and powering need support-aware budgets *before* allocating a large expansion. The old generic recommendation to split resource budgets is already in the register; the additional information here is the exact new compatibility call that can defeat a sparsity assumption. [R4]

11.2 A02: interrupted loading can bypass context cleanup

The Mathics loader inserts the compatibility context while definitions are read and removes it at the end of the normal load path. The inspected path does not wrap the entire interval in exceptional cleanup. If evaluation is interrupted after insertion and before the final cleanup, that final statement is not executed. The same source also enters package/private contexts during loading. [R7, R14]

This is a static exception-safety concern, not a captured interrupted-kernel failure. Its possible consequence is broader than a partially installed function: subsequent user input could resolve names

through a context intended only for package implementation. Unchanged System definitions do not prove that the user's name-resolution environment is unchanged.

A loader test should inject interruption after each compatibility phase and check the caller's context, context path, subsequent name resolution, and ability to reload. A repair needs structured cleanup and a clear partial-load state. Restoring only the visible \$ContextPath list should not be assumed to unwind every internal Begin/End stack effect. The correct mechanism must be tested in both runtimes rather than supplied as an unverified one-line "finally" emulation.

12 An executable, exact-rational oracle for the lower Lambert branch

The companion `lambert_minus_one.py` addresses the practical testing need exposed by F01. It encloses $W_{-1}(-q)$ for rational q satisfying $0 < q < e^{-1}$, using only Python integer and Fraction arithmetic in its proof-producing computation. SymPy and mpmath are used only by the independent test file, not by the enclosure algorithm.

12.1 Replace the ill-behaved conversion boundary with a monotone equation

Set $w = -t$. The branch $w < -1$ corresponds to $t > 1$, and

$$we^w = -q \iff te^{-t} = q \iff H_q(t) := t - \log t + \log q = 0.$$

For $t > 1$,

$$H'_q(t) = 1 - \frac{1}{t} > 0.$$

Moreover, $H_q(1) = 1 + \log q < 0$ precisely on the desired rational-input domain, and $H_q(t) \rightarrow +\infty$ as $t \rightarrow \infty$. Thus the root is unique. Certifying the sign of H_q at two rational endpoints gives a certificate without evaluating Lambert W at all.

12.2 A rational logarithm enclosure

For rational $r > 0$, write $r = 2^k m$ with integer k and $1 \leq m < 2$. Put

$$a = \frac{m-1}{m+1}, \quad 0 \leq a < \frac{1}{3}.$$

The identity

$$\log m = 2 \operatorname{artanh} a = 2 \sum_{j=0}^{N-1} \frac{a^{2j+1}}{2j+1} + R_N$$

has the explicit tail bounds

$$0 \leq R_N \leq \frac{2a^{2N+1}}{(2N+1)(1-a^2)}.$$

Indeed, every omitted denominator is at least $2N+1$, and the remaining powers form a geometric series. The same formula at $m = 2$ gives a bound for $\log 2$ using $a = 1/3$.

To avoid accumulating enormous unrelated rational denominators in a long sum, the implementation rounds each nonnegative term outward onto the dyadic lattice $2^{-B}\mathbb{Z}$. For a rational term p/d , use

$$\frac{\lfloor 2^B p/d \rfloor}{2^B} \leq \frac{p}{d} \leq \frac{\lceil 2^B p/d \rceil}{2^B}.$$

Add the outward-rounded tail to the upper endpoint. Combine with $k \log 2$, reversing interval endpoint roles when $k < 0$. Every rounding operation is integer division, so there is no assumed floating-point accuracy in the proof.

Proposition 1. *Suppose rational endpoints $1 \leq a < b$ have certified bounds $\overline{H}_q(a) < 0 < \underline{H}_q(b)$. Then*

$$W_{-1}(-q) \in [-b, -a],$$

and the enclosed real branch value is unique.

Proof. Continuity and the opposite endpoint signs imply a zero of H_q in (a, b) . Strict positivity of H'_q for $t > 1$ gives uniqueness. The algebraic equivalence above maps this zero to the branch $w = -t < -1$. Negating the interval reverses its endpoints. $\square$

The algorithm first certifies $H_q(1) < 0$, doubles an upper endpoint until its sign is positive, and then bisectionally. When an interval evaluation straddles zero, it increases the internal dyadic precision rather than guessing the sign. It stops when the root interval has width at most 10^{-d} for the requested d . Explicit resource limits raise `ProofBudgetExceeded`; they do not return an uncertified endpoint.

12.3 Executed checks and limitations

Seven independent Python tests passed. They include logarithm bounds for seven positive rationals, five lower-branch enclosures at 35-decimal-place absolute interval width, invalid-input and resource-exhaustion checks, the SymPy argument-order counterexamples, a formal logarithmic inverse coefficient check, and a first-match operation-count model. The five Lambert inputs were

$$q \in \left\{ \frac{1}{10}, \frac{1}{1000}, 10^{-30}, \frac{9}{25}, \frac{367879}{10^6} \right\}.$$

The last value is close to the branch point but still on the admitted side. The test oracle used `mpmath` at 120 decimal digits and checked that its reference value lay inside the rational enclosure. The certificate itself consists of rational endpoints and strict rational bounds for their H_q values; it does not depend on trusting that numerical oracle.

This is a reference implementation, not a proof-assistant-verified program and not a complete complex Lambert implementation. It handles rational real inputs on the lower branch, not arbitrary symbolic parameters, the irrational branch endpoint as an exact input, or other complex branches. A Mathics integration still needs a representation and conversion policy. The algorithm provides a concrete independently checkable piece of that work without pretending the integration has already happened.

13 Development recommendations that follow from the new findings

13.1 Treat compatibility adapters as a small semantic runtime

The compatibility layer is now large enough that a loose collection of substitutions is difficult to audit by inspection. The next useful step is not a new all-purpose intermediate representation, already proposed in prior reviews, but a much smaller *adapter contract manifest*. For each adapter, record the foreign/runtime operation, supported call shapes, evaluation attributes, fallback behavior, rewritten consumers, expected rewrite count, and independent control probes.

For example, the `ProductLog` entry needs separate input and output argument order, accepted arities, branch domain, derivative semantics, and specialization behavior. The `Map` entry needs empty-input alias reuse and an inventory of public/private/compatibility definitions. The `Limit` entry needs an approach direction. The `FirstPosition` entry needs its lazy default and short-circuit behavior, not just the returned index.

Late `DownValue` rewrites should report whether their intended patterns were actually found. A refactoring can otherwise make a rewrite match zero sites without causing a load error. A successful Wolfram definition-preservation comparison cannot detect that failure: the Mathics-only transformation intentionally does not run on Wolfram. This is an additional acceptance dimension beyond preserving the native kernel's definitions.

13.2 Add mutation tests at the semantic boundary

A small, deliberately adversarial suite is more valuable here than a large set of another ordinary quadratic inverse. Inject these faults one at a time: reverse the Lambert tuple; omit its reverse conversion; bypass the principal normalization; disable one safe-map call site; force a failed package load; replace an early-exit search by full traversal; or remove an explicit limit direction. Each mutation should make a targeted probe fail.

The important oracle question is independence. A test comparing the package against the same foreign conversion path can agree on the same wrong result. Use defining equations and selected real-branch intervals for Lambert values, exact coefficient identities for elementary inverse cases, input-reuse checks for alias-sensitive primitives, and phase-success markers for the runner. A mutation suite also exposes negative tests that pass because a helper remained unresolved rather than because a forbidden conclusion was rejected correctly.

13.3 Record semantic state at value boundaries

The current construction-time metadata is useful, but F01 shows that mathematical validity can be lost when a stored symbolic expression is specialized. A targeted improvement is to associate special-function payloads with a backend capability requirement: the exact expression can be retained symbolically, but numerical application must either use an admitted evaluator, return an explicit unsupported-evaluation failure, or produce a value together with an independently checked defining-equation and branch result.

This recommendation is narrower than the earlier general request for checked numerical evaluation. It identifies a concrete capability dependency: nonprincipal Lambert input/output convention in Mathics. It also explains why a numerical check should not silently use the same defective evaluator for

both the approximation and the supposed reference.

13.4 Budget observable work, not only final support

The current jet multiplication already prunes candidate products at a cutoff. The new performance work belongs at the compatibility boundary: first-match predicate counts, dense coefficient slots, conversion time, and proof attempts. These should be measured separately from the number of retained asymptotic terms.

For F05, record candidate predicate counts and total simplifier calls on repeated early matches. For A01, record polynomial degree, nonzero support, and peak allocation before requesting a dense conversion. For special-function conversion, record whether a request was exact, inexact, or symbolic and whether it crossed SymPy or mpmath. Do not compare a cold Python interpreter startup with a warm Wolfram coefficient loop and call the ratio an algebraic speed difference.

13.5 Sequence the work with explicit acceptance gates

The first change set should contain F01 containment, F02's empty-index guard, and F03's load gate, with the associated focused regressions on both entry points. The second should add the bounded first-match specialization and audit all internal limit directions. The third should introduce the adapter manifest, rewrite-count checks, interruption tests, and sparse-conversion stress controls.

Only after those gates should a broader compatibility claim be updated. The release record should name the exact source commit, interpreter/scanner/SymPy/Python versions, entry point, executed case set, and unresolved categories. Existing historical Wolfram preservation receipts should remain attached to their own snapshots rather than being promoted to evidence for later merged code.

14 Novelty crosswalk and exclusions

Item	Closest prior material	Additional contribution here
F01	Mathics compatibility warning about nonprincipal numerical Lambert evaluation; D05/X05 on numerical checking/certification	Exact SymPy conversion, inherited numeric arity, reverse-bridge requirement, package-generated value path, and an executed independent rational lower-branch oracle.
F02	New empty-list workaround and its documentation	Public InverseExpansionCoefficient is outside the private-only rewrite; valid empty multi-index plus post-call caller-input reuse.
F03	V02 on earlier test-runner acceptance problems	Specific setup invariant missing in the newer portable suite; a selected primitive can pass after Get returned failure. Not a repeat of empty discovery or old aborted-suite accounting.
F04	C18 and D06 on direction/coordinate conventions	Default semantics of the scoped Mathics Limit primitive; a two-sided proposition becomes a from-below request. No newly reproduced public metadata error claimed.

Item	Closest prior material	Additional contribution here
F05	P08 on repeated proof and normalization work	Exact all-positions adapter mechanism, symbolic bucket-search consumer, early-match workload, and a bounded repair that avoids an unsupported <code>Mathics Position</code> form.
A01	P06 on resource budgets; earlier dense-export findings	Identifies a separate <code>Mathics</code> coefficient-extraction conversion. Kept advisory because a public stress path was not established.
A02	Normal load/reload isolation and definition-preservation checks	Exceptional exit between context insertion and cleanup; interruption and context-stack acceptance tests. No observed interrupted-kernel result claimed.
Discrete	Existing native/discrete extension proposals	Distinguishes continuous germs from integer-sequence filters and from log-Fourier modes; a conversion precondition, not a reissued feature request.

The article deliberately does not reissue the old optional `SeriesData` allocation bug, previously tracked remainder-degree and assumption-scope bugs, native option/default routing issues, the existing public coefficient-option override problem, generic association validation, old refinement precision problems, or the broad integration/complex-transseries/uniform-parameter wish list. Those remain in the repository's register with their established status. [[R4](#), [R5](#)]

The anti-duplication claim is bounded: the consolidated registers, wave-3 intake, report index, and an original relevant review were inspected. This is not a declaration that every sentence in every historical PDF was compared mechanically. The new items are presented with their exact delta so a maintainer can merge them into the existing register without inflating the defect count.

15 Reproduction guide and archive contents

The article source is self-contained apart from standard TeX packages and installed fonts. Its PDF was generated from that source. The code directory contains the standalone exact-rational Lambert enclosure, its independent checks, a fresh-kernel probe runner, and the deliberate failed-load fixture. The patches directory contains focused edit proposals, not silently applied modifications. The evidence directory separates actual Python results from the explicit unrun kernel status and the source/novelty ledger.

To reproduce the independent checks:

```
python -m pip install -r code/requirements.txt
python code/independent_checks.py
python code/lambert_minus_one.py 1/10 --digits 35
```

The enclosure script itself requires only the Python standard library. The requirements are for independent SymPy/mpmath cross-checks.

To execute the currently unrun `Mathics` probes against a local checkout:


```
python code/run_probes.py --repo C:\src\Asymptotic \
--python C:\venv\Scripts\python.exe --entry modular
python code/run_probes.py --repo C:\src\Asymptotic \
--python C:\venv\Scripts\python.exe --entry standalone
```

Use the repository's own pinned Mathics requirements for that interpreter environment. Backslashes above denote paths; on PowerShell put the command on one line or use its backtick continuation rather than a Unix trailing backslash. The `--wolfram wolfram.exe` option selects a local Wolfram executable instead. The probe runner performs no downloads and makes no source edits. A Fail on the pinned snapshot can be the expected characterization of an unfixed finding; it is not automatically a failure of the probe infrastructure.

For F03 specifically, use the upstream runner with the supplied invalid source fixture and select `primitive-check-is-unpolluted`. After the proposed load gate is applied, this invocation must produce an error, not a successful selected-case receipt. This probe concerns the upstream runner; the bundle's runner already enforces its own failed-load gate.

Rebuild the article with three passes of XeLaTeX to settle the contents and table widths:

```
cd article
xelatex -interaction=nonstopmode -halt-on-error asymptotic-review.tex
xelatex -interaction=nonstopmode -halt-on-error asymptotic-review.tex
xelatex -interaction=nonstopmode -halt-on-error asymptotic-review.tex
```

No checksum files, font files, private account data, repository commits, or remote changes are part of this archive.

16 Conclusion

The package's principal contribution remains its explicit calculus of real generalized expansions and retained error information, especially for inverse functions outside a simple rational-power lattice. Native Wolfram functionality is broader, but breadth alone does not replace this representation and workflow. The correct development direction is complementary: retain the package's useful contracts, delegate only where the native contract is explicit, and avoid calling a different backend's unresolved or misinterpreted expression a completed calculation.

For Mathics, the current engineering is substantial and its own documentation is appropriately cautious about completeness. The next correctness gains are concentrated at a few well-defined boundaries. Exact symbolic Lambert conversion needs a bidirectional repair or containment; public empty-list arguments need the same protection as private jets; a successful test must imply successful setup; and first-match and limit-direction semantics must not be weakened by apparently convenient adapters.

Those changes have unusually good regression targets. They can be checked with small cases, explicit branch equations, observable predicate counts, and deliberate fault injection. The supplied rational W_{-1} oracle and unrun kernel probes make the proposed work concrete without presenting an unexecuted Mathics integration as finished.

References

- [R1] Vladimir Reshetnikov. *Asymptotic repository, README*. Reviewed commit 7d1bc832895c. <https://github.com/VladimirReshetnikov/Asymptotic/blob/7d1bc832895cc90a9b2a978b7b7684acab908bd2/README.md>.
- [R2] Repository commit metadata. <https://github.com/VladimirReshetnikov/Asymptotic/commit/7d1bc832895cc90a9b2a978b7b7684acab908bd2>.
- [R3] *Code review reports index*. <https://github.com/VladimirReshetnikov/Asymptotic/blob/7d1bc832895cc90a9b2a978b7b7684acab908bd2/external-reports/code-review/README.md>.
- [R4] *Consolidated implementation status*. https://github.com/VladimirReshetnikov/Asymptotic/blob/7d1bc832895cc90a9b2a978b7b7684acab908bd2/docs/development/CODE_REVIEW_STATUS.md.
- [R5] *Wave-3 intake and attributed finding ledger*. https://github.com/VladimirReshetnikov/Asymptotic/blob/7d1bc832895cc90a9b2a978b7b7684acab908bd2/docs/development/WAVE_3_INTAKE.md.
- [R6] *Review 23: scope, findings, evidence, and unrun native probes*. <https://github.com/VladimirReshetnikov/Asymptotic/blob/7d1bc832895cc90a9b2a978b7b7684acab908bd2/external-reports/code-review/wave-3/code-review-23/README.md>.
- [R7] *Core package implementation*. In particular: bootstrap; jetMerge; precision triples; object application; public InverseExpansionCoefficient; final loader cleanup. <https://github.com/VladimirReshetnikov/Asymptotic/blob/7d1bc832895cc90a9b2a978b7b7684acab908bd2/src/Kernel/AsymptoticAnalysis.wl>.
- [R8] *Kernel module map*. <https://github.com/VladimirReshetnikov/Asymptotic/blob/7d1bc832895cc90a9b2a978b7b7684acab908bd2/src/Kernel/README.md>.
- [R9] *NativeCompatibility.wl*. Held request dispatch, native contracts and evaluation status. <https://github.com/VladimirReshetnikov/Asymptotic/blob/7d1bc832895cc90a9b2a978b7b7684acab908bd2/src/Kernel/NativeCompatibility.wl>.
- [R10] *LambertInverse.wl*. <https://github.com/VladimirReshetnikov/Asymptotic/blob/7d1bc832895cc90a9b2a978b7b7684acab908bd2/src/Kernel/LambertInverse.wl>.
- [R11] *CorePerturbation.wl*. Automatic inverse selection and exact-core construction. <https://github.com/VladimirReshetnikov/Asymptotic/blob/7d1bc832895cc90a9b2a978b7b7684acab908bd2/src/Kernel/CorePerturbation.wl>.
- [R12] *FourierCoefficients.wl*. <https://github.com/VladimirReshetnikov/Asymptotic/blob/7d1bc832895cc90a9b2a978b7b7684acab908bd2/src/Kernel/FourierCoefficients.wl>.
- [R13] *Running AsymptoticAnalysis in Mathics3*. Supported examples, limitations and scoped preservation receipts. <https://github.com/VladimirReshetnikov/Asymptotic/blob/7d1bc832895cc90a9b2a978b7b7684acab908bd2/docs/Mathics/COMPATIBILITY.md>.
- [R14] *MathicsCompatibility.wl*. Scoped evaluator, lookup, and first-position adapters. <https://github.com/VladimirReshetnikov/Asymptotic/blob/7d1bc832895cc90a9b2a978b7b7684acab908bd2/src/Kernel/MathicsCompatibility.wl>.
- [R15] *MathicsSimplification.wl*. <https://github.com/VladimirReshetnikov/Asymptotic/blob/7d1bc832895cc90a9b2a978b7b7684acab908bd2/src/Kernel/MathicsSimplification.wl>.
- [R16] *MathicsAlgebra.wl*. <https://github.com/VladimirReshetnikov/Asymptotic/blob/7d1bc832895cc90a9b2a978b7b7684acab908bd2/src/Kernel/MathicsAlgebra.wl>.

- [R17] *MathicsCoreFunctions.wl*. Principal Lambert normalization and nonprincipal passthrough. <https://github.com/VladimirReshetnikov/Asymptotic/blob/7d1bc832895cc90a9b2a978b7b7684acab908bd2/src/Kernel/MathicsCoreFunctions.wl>.
- [R18] *MathicsLists.wl*. Private-only DownValue installer. <https://github.com/VladimirReshetnikov/Asymptotic/blob/7d1bc832895cc90a9b2a978b7b7684acab908bd2/src/Kernel/MathicsLists.wl>.
- [R19] *Empty-list mapping in Mathics*. <https://github.com/VladimirReshetnikov/Asymptotic/blob/7d1bc832895cc90a9b2a978b7b7684acab908bd2/docs/Mathics/LISTS.md>.
- [R20] *MathicsSpecialFunctions.wl*. <https://github.com/VladimirReshetnikov/Asymptotic/blob/7d1bc832895cc90a9b2a978b7b7684acab908bd2/src/Kernel/MathicsSpecialFunctions.wl>.
- [R21] *MathicsTests.wl*. Setup and foundational portable cases, especially lines 1–110. <https://github.com/VladimirReshetnikov/Asymptotic/blob/7d1bc832895cc90a9b2a978b7b7684acab908bd2/validation/MathicsTests.wl>.
- [R22] *run_mathics_tests.py*. Case selection and strict per-process result protocol. https://github.com/VladimirReshetnikov/Asymptotic/blob/7d1bc832895cc90a9b2a978b7b7684acab908bd2/validation/run_mathics_tests.py.
- [R23] *Mathics compatibility workflow*. <https://github.com/VladimirReshetnikov/Asymptotic/blob/7d1bc832895cc90a9b2a978b7b7684acab908bd2/.github/workflows/mathics.yml>.
- [M1] Mathics3. Version 10.0.1 reference. <https://github.com/Mathics3/mathics-core/tree/f6a987e704313e37b2b46ed4fe0b61aab108b58a>.
- [M2] Mathics3 10.0.1. *Exponential integral and special functions*, ProductLog class. <https://github.com/Mathics3/mathics-core/blob/f6a987e704313e37b2b46ed4fe0b61aab108b58a/mathics/builtin/specialfns/expintegral.py>.
- [M3] Mathics3 10.0.1. *Built-in base classes*, SympyFunction and MPMathFunction, particularly lines 738–858. <https://github.com/Mathics3/mathics-core/blob/f6a987e704313e37b2b46ed4fe0b61aab108b58a/mathics/core/builtin.py>.
- [M4] Mathics3 10.0.1. *List element operations*, Position implementation, approximately lines 1278–1346. <https://github.com/Mathics3/mathics-core/blob/f6a987e704313e37b2b46ed4fe0b61aab108b58a/mathics/builtin/list/eol.py>.
- [W1] Wolfram Research. *Series*. Current documentation, accessed September 9, 2026. <https://reference.wolfram.com/language/ref/Series.html>.
- [W2] Wolfram Research. *SeriesData*. Current documentation. <https://reference.wolfram.com/language/ref/SeriesData.html>.
- [W3] Wolfram Research. *InverseSeries*. Current documentation. <https://reference.wolfram.com/language/ref/InverseSeries.html>.
- [W4] Wolfram Research. *Asymptotic*. Current documentation. <https://reference.wolfram.com/language/ref/Asymptotic.html>.
- [W5] Wolfram Research. *DiscreteAsymptotic*. Current documentation. <https://reference.wolfram.com/language/ref/DiscreteAsymptotic.html>.
- [W6] Wolfram Research. *Asymptotics guide*. Current built-in function inventory. <https://reference.wolfram.com/language/guide/Asymptotics.html>.
- [W7] Wolfram Research. *Limit*, default real direction and one-sided conventions. <https://reference.wolfram.com/language/ref/Limit.html>.
- [S1] SymPy. *Elementary functions*, LambertW, version 1.14.0 documentation. <https://docs.sympy.org/latest/modules/functions/elementary.html#sympy.functions.elementary.exponential.LambertW>.